校招AI应用工程师·一个月任务清单

🎯 **任务主题：企业级 AI 模型智能路由与 Token 精细化管控系统**

背景：企业日均 Token 消耗 3~4 万美金。需要一套**可部署运行的系统**，实现开发任务自动推荐模型、Token 消费透明可审计、优化策略可执行。这不是调研报告，是**可以上线的产品**。

总体交付目标

💡 **管控平台（Web）**

- 输入任务描述 → 自动推荐模型组合
- 实时 Token 消费看板
- 优化建议与一键应用
- 路由规则管理后台

💡 **CLI / SDK 工具集**

- ai-router analyze：任务→模型推荐
- ai-router audit：Token 消费审计
- ai-router optimize：Prompt 优化建议
- 可集成到 CI/CD 流程

同时输出：**《Claude 模型 Token 优化最佳实践手册》** + **《企业 Token 管控落地 SOP》**

核心能力矩阵

本任务考核的**不是"会不会用 AI"**，而是能否**驾驭 AI 解决企业级问题**：

能力维度	具体要求	在任务中的体现
业务理解力	能将"省 Token"翻译成可量化的技术指标	路由规则设计、成本模型建模
架构设计力	能设计可扩展、可维护的系统架构	路由引擎架构、插件化模型接入
工程落地力	代码可运行、可测试、可部署	Docker 化部署、自动化测试、CI/CD
数据驱动力	用数据说话，而非拍脑袋	Benchmark 体系、A/B 对比、成本回归分析

产品交付力	产出物是"产品"而非"Demo"	UI/UX 可用性、文档完整性、SOP 可操作性

第 1 周：深度调研 + 系统架构设计

> 📐 **目标**：完成竞品调研、技术选型、架构设计，输出《系统设计文档》

1.1 深度竞品与技术调研

序号	子任务	交付标准（企业级）
1.1a	调研 6+ 大模型的定价结构（按 input/output token、缓存命中、批处理折扣等维度），建立**统一成本模型**	成本对比矩阵表 + 归一化定价公式 + 不同调用模式下的月成本模拟
1.1b	深度研究 Claude Prompt Caching、Extended Thinking、Batch API 等**原生降本机制**的原理和适用场景	每种机制的实测数据（命中率、延迟、质量影响）+ 决策树：什么场景用什么机制
1.1c	调研业界 Token 管控方案（OpenRouter、LiteLLM、Claude-tap、Helicone 等），做**功能对比矩阵**	对比矩阵 + Gap 分析：现有方案缺什么、我们的差异化在哪
1.1d	设计**任务分类体系**：将开发任务按复杂度、上下文长度、推理强度、代码生成量等维度分级	分类 Taxonomy + 每个级别的典型任务示例 + 对应推荐模型和预估 Token 消耗

1.2 系统架构设计

序号	子任务	交付标准（企业级）
1.2a	设计**模型路由引擎**架构：输入任务描述 → 任务分类 → 模型推荐 → 成本预估 → 输出路由决策	架构图 + 核心模块接口定义（OpenAPI Spec）+ 路由决策算法伪代码
1.2b	设计**Token 消费采集与审计**架构：API Proxy / Middleware 层拦截、消费归因（按人/部门/项目/任务类型）	数据流图 + 存储 Schema + 归因模型设计

1.2c	设计**技术选型方案**：前端框架、后端语言/框架、数据库、部署方式	选型对比表 + 决策依据 + 技术风险与缓解方案

📦 **第 1 周交付物**：《系统设计文档》（含竞品调研、架构设计、技术选型、接口定义、任务分类体系）

第 2 周：核心引擎开发

> 💡 **目标**：完成路由引擎 + CLI 工具的核心功能，可通过自动化测试验证

2.1 模型路由引擎（核心）

序号	子任务	交付标准（企业级）
2.1a	实现**任务分析器**：接收自然语言任务描述，提取特征（复杂度、领域、上下文需求、输出格式要求等）	支持 10+ 维特征提取 + 50 条测试用例覆盖率 ≥ 80% + 分析延迟 < 500ms
2.1b	实现**路由决策引擎**：基于任务特征 + 成本模型 + 质量要求，输出最优模型组合	支持规则引擎 + 可配置权重 + 路由决策可解释（输出推荐理由）+ 支持 fallback 策略
2.1c	实现**成本预估器**：给定任务 + 模型组合，预估 Token 消耗和费用	预估偏差 ≤ 30%（基于历史数据校准）+ 支持多模型组合的成本叠加计算

2.2 CLI 工具集

序号	子任务	交付标准（企业级）

2.2a	实现 **ai-router analyze** 命令：输入任务描述，输出模型推荐 + 成本预估 + 路由理由	代码块
		`$ ai-router analyze "重构用户登录模块，需要支持 OAuth2 和 MFA"`
		`→ 任务分类：中等复杂度 | 后端重构 | 需要代码理解+生成`
		`→ 推荐：Sonnet 4.6（规划+审查）+ Haiku 4.5（批量生成）`
		`→ 预估：45K tokens，~$0.32`
		`→ vs 纯 Opus：节省 78%，质量影响：低`

		6 → 建议：启用 Prompt Caching（预估额外节省 40%）
2.2b	实现 **ai-router audit** 命令：审计指定时间范围的 Token 消费，识别浪费模式	输出消费汇总 + 按人/项目/任务类型归因 + Top 5 浪费模式 + 优化建议（带预估节省金额）
2.2c	实现 **ai-router optimize** 命令：分析当前 Prompt/上下文，给出具体优化建议和预估节省比例	至少支持 5 种优化策略（缓存/压缩/分层/批处理/降级）+ 每种策略的预估节省比例 + 可一键应用

2.3 Claude 专项优化模块

🔬 **核心要求**：仅使用 Claude 模型家族时，如何实现最优成本效率？

序号	优化策略	交付标准（企业级）
2.3a	**Prompt Caching 策略**：实现缓存命中率监控 + 自动识别可缓存的 System Prompt / 工具定义 / 上下文片段	缓存命中率 Dashboard + 自动建议缓存策略 + 实测节省数据（目标 ≥ 50% 场景可缓存）
2.3b	**模型分级使用策略**：Opus/Sonnet/Haiku 三级任务的自动识别与分配规则	分级规则引擎 + 实测：在保证质量前提下，Opus 使用占比从 100% 降至 ≤ 15%
2.3c	**上下文精简引擎**：自动检测冗余上下文、建议裁剪策略、衡量裁剪前后质量变化	自动检测 + 裁剪建议 + A/B 质量对比数据 + 目标：上下文 Token 减少 30% 且质量下降 ≤ 5%

📦 **第 2 周交付物**：可运行的路由引擎 + CLI 工具集 + Claude 专项优化模块 + 自动化测试套件（覆盖率 ≥ 80%）

第 3 周：管控平台 + 数据驱动优化

📊 **目标**：完成 Web 管控平台，接入真实数据，实现可视化运营

3.1 管控平台核心功能

序号	功能模块	交付标准（企业级）
3.1a	**任务路由工作台**：输入开发任务 → 可视化展示推荐模型、成本预估、替代方案对比	响应时间 ≤ 2s + 支持批量任务分析 + 路由历史记录可追溯
3.1b	**消费看板**：实时 Token 消费大屏，支持按部门/项目/人员/模型/任务类型多维下钻	实时刷新 ≤ 5min + 趋势图 + 预算预警（超 80% 告警）+ 同比/环比分析
3.1c	**优化建议中心**：基于消费数据自动生成优化建议，按预估节省金额排序	每条建议含：问题描述 + 优化方案 + 预估节省金额 + 一键应用按钮 + 应用后效果追踪
3.1d	**路由规则管理**：管理员可配置/调整路由规则、模型权重、预算限额	规则版本管理 + 灰度发布 + 回滚机制 + 变更审计日志

3.2 Benchmark 与数据驱动

序号	子任务	交付标准（企业级）
3.2a	构建**自动化 Benchmark 套件**：覆盖代码生成、代码审查、重构、文档编写、Bug 分析等 5+ 任务类型	每类任务 ≥ 20 条测试用例 + 评估维度（正确性、完整性、延迟、成本）+ 自动回归
3.2b	基于 Benchmark 数据，输出**任务-模型适配矩阵**：哪类任务用哪个模型性价比最高	矩阵可视化 + 置信度标注 + 定期自动更新机制
3.2c	实现**路由效果回测**：用历史数据验证路由决策的质量，量化节省金额和质量影响	回测报告：纯 Opus vs 路由后混合方案的 成本/质量/延迟 三维对比

3.3 工程化要求

工程要求	交付标准
Docker 化部署	docker-compose up 一键启动全部服务（前端 + 后端 + 数据库）
API 文档	所有 API 有 OpenAPI Spec，可在线交互测试
自动化测试	单元测试覆盖率 ≥ 80% + 集成测试覆盖核心链路

日志与监控	结构化日志 + 关键指标 Prometheus metrics + Grafana 看板

📦 **第 3 周交付物**：可部署的管控平台 + Benchmark 套件 + 任务-模型适配矩阵 + Docker Compose 部署方案

第 4 周：最佳实践手册 + 落地 SOP + 答辩

> 📖 **目标**：输出可直接在企业内部推广使用的最佳实践和 SOP，并完成答辩

4.1 《Claude 模型 Token 优化最佳实践手册》

章节	内容要求	交付标准（企业级）
第 1 章	Claude 模型家族全景：Opus/Sonnet/Haiku 能力边界与成本结构	含实测数据，非官网搬运；每个模型给出"适合做什么/不适合做什么"
第 2 章	Prompt Caching 深度指南：原理、适用场景、配置方法、命中率优化	含代码示例 + 不同场景的命中率实测 + 常见踩坑清单
第 3 章	上下文工程：如何精简上下文而不损失质量	5+ 种精简策略 + 每种策略的实测效果 + 自动裁剪工具使用指南
第 4 章	模型分级策略：什么任务用 Opus、什么用 Sonnet、什么用 Haiku	决策树 + Benchmark 数据支撑 + 边界 Case 处理方案
第 5 章	Agent 场景优化：Claude Code / Cursor 等 Harness 的 Token 优化	Harness 层 Token 消耗拆解 + 各环节优化策略 + 实测节省数据
第 6 章	成本监控与预警：如何建立企业级 Token 消费透明度	监控指标体系 + 预警规则 + 管控平台使用指南

4.2 《企业 Token 管控落地 SOP》

章节	内容要求	交付标准（企业级）

SOP-1	**开通流程**：新员工/新部门如何接入管控系统	Step-by-step 操作手册 + 截图 + 常见问题 FAQ
SOP-2	**日常使用规范**：什么场景走路由推荐、什么场景可以手动指定模型	使用规范 + 例外审批流程 + 违规处理机制
SOP-3	**成本复盘**：周/月 Token 消费复盘会议模板	数据看板模板 + 复盘会议议程 + Action Item 跟踪表
SOP-4	**路由规则迭代**：如何基于数据持续优化路由规则	规则迭代流程 + 灰度发布 Checklist + 回滚预案
SOP-5	**管理层汇报**：给管理层提供 Token 消费数据支撑	汇报模板 + 关键指标定义 + 部门间对比分析模板

4.3 答辩

答辩环节	要求
系统演示（15 min）	现场演示管控平台和 CLI 工具，用真实开发任务走完全流程
数据汇报（10 min）	展示 Benchmark 数据、成本节省数据、路由效果回测数据
技术深潜（10 min）	讲解路由引擎核心算法、架构决策的 Trade-off
Q&A（15 min）	由 3+ 位面试官（含技术主管和业务负责人）提问

📦 **第 4 周交付物**：《Claude Token 优化最佳实践手册》+《企业 Token 管控落地 SOP》+ 答辩演示

最终交付物清单

序号	交付物	形态	验收标准
1	系统设计文档	飞书文档 / Markdown	架构评审通过
2	模型路由引擎	Python/Go 代码仓库	测试覆盖率 ≥ 80%
3	CLI 工具集（ai-router）	可安装 CLI 包	3 个核心命令均可用
4	Web 管控平台	前后端代码仓库	Docker Compose 一键部署

5	Claude 专项优化模块	代码 + 配置 + 文档	实测节省数据支撑
6	自动化 Benchmark 套件	测试框架 + 数据集	5+ 任务类型 × 20+ 用例
7	Claude Token 优化最佳实践手册	飞书文档（6 章）	含实测数据，非搬运
8	企业 Token 管控落地 SOP	飞书文档（5 个 SOP）	可直接在企业推广
9	答辩演示	PPT + 现场演示	3+ 面试官评审通过

难度评估与应对策略

维度	难度	挑战点	应对策略
架构设计	⭐⭐⭐⭐⭐	路由引擎需要兼顾准确性与灵活性	导师第 1 周做架构 Review，提供设计反馈
工程实现	⭐⭐⭐⭐⭐	4 周内完成前后端 + CLI + 部署	优先保证核心链路可用，非核心功能可简化
数据驱动	⭐⭐⭐	需要足够的测试数据支撑 Benchmark	导师提供脱敏历史调用数据 + 模拟数据生成器
业务理解	⭐⭐⭐⭐⭐	任务分类体系需要贴合真实开发场景	安排 1 天跟随导师参与真实项目，理解业务上下文
文档产出	⭐⭐⭐	最佳实践手册需要实测数据而非搬运	提供 API 额度用于实测 + 第 2 周开始积累数据

✅ **整体难度评估：高但有支撑**

- 这是一个**准高级工程师**级别的考核任务，对标企业真实项目交付标准
- 4 周时间紧凑，但每周有独立里程碑，导师每周做 Review
- 核心链路（路由引擎 + CLI + 看板）必须完成，非核心功能可裁剪
- 最终交付物可直接用于企业内部推广，**不是玩具项目**

评分体系（总分 100 分）

评分维度	权重	评分要点
架构设计	15%	架构合理性、可扩展性、接口设计规范性
路由引擎质量	20%	推荐准确率、成本预估偏差、决策可解释性
工程实现	20%	代码质量、测试覆盖率、Docker 部署可用性
数据驱动能力	15%	Benchmark 设计合理性、数据说服力、回测可信度
文档与 SOP	15%	手册实用性、SOP 可操作性、数据是否来自实测
答辩表现	15%	系统演示流畅度、技术深度、Q&A 应变能力

导师支撑计划

时间	支撑动作	内容
Day 1	项目启动会	明确目标、介绍业务背景、提供参考资料和 API 额度
Week 1 末	架构 Review	评审系统设计文档，给出改进建议；确认技术选型合理
Week 2 中	代码 Review	Review 核心模块代码质量，指导工程实践
Week 2 末	中期检查	确认核心引擎可用，评估进度风险，必要时裁剪范围
Week 3 末	平台验收	Docker Compose 部署验收，确认平台核心功能可用
Week 4 中	文档 Review	评审最佳实践手册和 SOP，确保可推广使用
Week 4 末	答辩评审	3+ 面试官参与答辩评审

补充研究方向（基于 CCH 网关与 Langfuse 实战）

> 🚀 **说明**：以下方向是对核心任务的**深化补充**，聚焦企业实际场景中的可观测性、上下文工程和 KV Cache 优化。建议融入 4 周计划中执行。

方向 A：Langfuse 可观测性体系建设

融入建议：Week 1 调研 + Week 3 平台集成

任务	内容	交付标准
A-1	**客户端识别与追踪**：在 Langfuse 中实现按客户端类型（Claude Code / Cursor / 自研 Agent / API 直调）的消费归因	通过 metadata 标记 client_type + user_id，Langfuse Dashboard 可按客户端维度聚合 Token 消耗、延迟、成本
A-2	**高消费模式挖掘**：从 trace 数据中自动识别浪费模式	输出"Top 10 浪费模式"报告：重复调用、过长上下文、未命中缓存、可降级未降级等，每条带预估节省金额
A-3	**Langfuse 与管控平台集成**：将 Langfuse 数据接入自研管控平台的消费看板	API 对接 + 数据同步 + 看板展示：实时消费、趋势、按客户端/用户/项目多维下钻

方向 B：上下文工程（Context Engineering）

融入建议：Week 1 调研 + Week 2 实现

任务	内容	交付标准
B-1	**上下文优先级框架设计**：设计分层上下文组装策略	优先级分层：System Prompt > 任务描述 > 相关文件 > 历史对话 > 辅助信息。每层有 Token 预算 + 淘汰策略 + 代码实现
B-2	**上下文压缩引擎**：实现 3+ 种压缩策略	支持：工具输出摘要化、对话历史压缩、冗余文件剔除。每种策略有 A/B 质量对比数据 + 成本节省数据
B-3	**上下文预算管理**：为不同任务类型设置最大上下文预算	任务分类 → 预算分配 → 超限自动裁剪 → 裁剪日志可审计。预算设置基于 Chroma Context Rot 研究的退化曲线

方向 C：CCH 网关 KV Cache 优化

融入建议：Week 1 调研 + Week 2 实现 + Week 3 监控

任务	内容	交付标准
C-1	**Prefix 稳定性优化**：在 CCH 网关层确保 System Prompt + Tool Definitions 前缀稳定	网关自动检测消息顺序变动 + "前缀锁定"机制 + 前缀变化率监控。目标：前缀稳定性 > 95%
C-2	**Cache-friendly 请求构造指南**：指导客户端如何构造缓存友好的请求	输出《客户端 KV Cache 最佳实践》文档：System Prompt 固定化、Tool 定义排序稳定、上下文追加而非替换
C-3	**KV Cache 命中率实时监控**：在网关层监控 cache_creation vs cache_read 比例	按客户端/用户/模型维度的缓存命中率看板 + 低命中率告警 + 自动诊断 miss 原因
C-4	**Safe-Trim 算法**：在保持 KV Cache 命中的同时，安全移除无用上下文	核心约束：只修改"缓存前缀之后"的内容；前缀部分只做"追加"不做"删除"。算法实现 + 实测数据（缓存命中率 vs 上下文精简率的平衡）

方向 D：Context Rot 与模型蠢笨区治理

融入建议：Week 1 调研 + Week 3 实现

任务	内容	交付标准
D-1	**Context Rot 编程场景复现实验**：在 5+ 编程任务上量化 Claude 模型的性能退化曲线	测试任务：代码生成、Bug 修复、重构、代码审查、文档生成。按输入长度梯度（1K/5K/10K/20K/50K/100K tokens）测试，输出退化曲线图 + 退化拐点
D-2	**蠢笨区自动检测**：实现"蠢笨区"实时监控与预警	指标设计：当输入 Token 超过退化阈值后，输出质量（正确率、指令遵循度）显著下降。实时监控 + 自动预警（建议开新会话或裁剪上下文）
D-3	**编程场景治理策略**：设计系统级治理方案	4 个策略：(1) 会话生命周期管理 (2) 上下文窗口预算管理 (3) 渐进式裁剪策略 (4) 多会话协作模式。每个策略有实现代码 + 实测效果数据
D-4	**干扰物影响研究**：量化编程场景中不同"干扰物"的影响	干扰物分类：无关文件内容、过时对话历史、冗余工具输出、重复代码片段。量化每种干扰物对任务质量的影响程度 + 设计过滤机制

补充方向的融入建议

> 📅 **时间分配建议**：4 个补充方向工作量较大，建议根据候选人能力**选做 2-3 个方向**，而非全部完成。

周次	核心任务	补充方向融入	交付物
Week 1	系统设计文档	调研 Langfuse、Context Engineering、KV Cache、Context Rot	设计文档补充 4 个方向的调研章节
Week 2	核心引擎 + CLI	实现上下文压缩引擎、Safe-Trim 算法	代码仓库补充对应模块
Week 3	管控平台 + Benchmark	集成 Langfuse、实现蠢笨区检测、Context Rot 实验	平台功能 + 实验报告
Week 4	最佳实践 + SOP + 答辩	将新方向的实践写入最佳实践手册	手册补充章节

更新后的最佳实践手册章节

章节	主题	内容要求
第 1 章	Claude 模型家族全景	Opus/Sonnet/Haiku 能力边界与成本结构（含实测数据）
第 2 章	Prompt Caching 深度指南	原理、适用场景、配置方法、命中率优化（含代码示例）
第 3 章	**上下文工程实战**（新增）	上下文优先级框架、压缩策略、预算管理、Context Rot 研究结论
第 4 章	**KV Cache 优化指南**（新增）	网关层 Prefix 稳定性、客户端 cache-friendly 构造、Safe-Trim 算法

第 5 章	模型分级策略	什么任务用 Opus/Sonnet/Haiku（决策树 + Benchmark 数据）
第 6 章	**蠢笨区治理**（新增）	Context Rot 编程场景实验、退化曲线、检测与预警、治理策略
第 7 章	Agent 场景优化	Claude Code / Cursor 等 Harness 的 Token 优化
第 8 章	**Langfuse 可观测性**（新增）	客户端追踪、消费归因、浪费模式挖掘、与管控平台集成
第 9 章	成本监控与预警	监控指标体系 + 预警规则 + 管控平台使用指南

更新后的评分体系（总分 100 分）

评分维度	权重	评分要点
架构设计	12%	架构合理性、可扩展性、接口设计规范性
路由引擎质量	15%	推荐准确率、成本预估偏差、决策可解释性
工程实现	15%	代码质量、测试覆盖率、Docker 部署可用性
数据驱动能力	12%	Benchmark 设计合理性、数据说服力、回测可信度
上下文工程能力（新增）	12%	上下文压缩效果、KV Cache 优化效果、蠢笨区治理效果
可观测性建设（新增）	10%	Langfuse 集成深度、客户端追踪精度、浪费模式挖掘价值
文档与 SOP	12%	手册实用性（9 章）、SOP 可操作性、数据是否来自实测
答辩表现	12%	系统演示流畅度、技术深度、Q&A 应变能力

(注：内容由 AI 生成，请谨慎参考)